[NOME DA INSTITUIÇÃO]
Disruptive Architectures: IoT, Big Data e IA

Pipeline de Dados com IoT e Docker

Ingestão, armazenamento e visualização de leituras reais de sensores de temperatura

Aluno: [SEU NOME]
Professor: [NOME DO PROFESSOR]
2026

1. Contextualização

Dispositivos IoT produzem leituras contínuas que, sozinhas, dizem pouco: valores soltos num arquivo não mostram tendências, falhas de coleta nem diferenças entre ambientes. Para virar informação útil, esses dados precisam passar por um caminho de ingestão, tratamento, armazenamento e visualização.

Este projeto constrói esse caminho de ponta a ponta com dados reais de um sensor de temperatura publicados no Kaggle. O resultado é um pipeline reproduzível: qualquer pessoa clona o repositório, sobe o banco com Docker, roda dois scripts e abre um dashboard interativo.

2. Objetivos

Objetivo geral: desenvolver um pipeline de dados IoT que processe, armazene e visualize leituras de temperatura usando Python, Docker, PostgreSQL e Streamlit.

Objetivos específicos:

- Analisar o schema real do dataset antes de qualquer implementação;
- Tratar os dados com Pandas (tipos, datas, nulos, duplicados e validação);
- Executar o PostgreSQL em contêiner Docker, com credenciais fora do código;
- Carregar os dados de forma idempotente e validar a carga direto no banco;
- Criar três views SQL que resumam os dados;
- Construir um dashboard interativo e extrair insights calculados dos dados.

3. Tecnologias utilizadas

Tecnologia	Papel no projeto
Python	Linguagem de todos os scripts do pipeline.
Pandas	Leitura do CSV, conversão de tipos, limpeza e validação.
Docker / Compose	Executa o PostgreSQL isolado, iniciado com `docker compose up -d`.
PostgreSQL 16	Banco relacional que guarda a tabela de leituras e as views.
SQLAlchemy + psycopg2	Conexão do Python com o banco e execução de SQL.
Streamlit	Interface web do dashboard, com filtros.
Plotly	Gráficos interativos com tooltips, zoom e legenda.
GitHub	Versionamento e publicação do código.
Kaggle / kagglehub	Origem do dataset e download automatizado.

4. Arquitetura do projeto

```
Kaggle (CSV) → Python/Pandas → PostgreSQL (Docker) → Views SQL → Streamlit + Plotly → Dashboard
```

1. **Extração:** `download_data.py` baixa o CSV com kagglehub para `data/`.
2. **Transformação:** `load_data.py` padroniza nomes, converte data e temperatura e remove registros problemáticos.
3. **Carga:** o mesmo script grava na tabela `temperature_readings` e cria as views.
4. **Validação:** `validate_db.py` executa consultas de conferência no banco.
5. **Visualização:** `dashboard.py` consulta as views e exibe os gráficos.

As credenciais ficam num arquivo `.env` (fora do Git); o repositório traz apenas o modelo `.env.example`.

5. Configuração do ambiente

Pré-requisitos instalados: Python, Git e Docker Desktop (com WSL 2, no Windows).
Passos:

```
copy .env.example .env          # e definir a senha
docker compose up -d            # sobe o PostgreSQL 16
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt # pandas, psycopg2-binary, SQLAlchemy, streamlit, plotly...
python src/load_data.py
python src/validate_db.py
streamlit run src/dashboard.py
```

6. Dataset

Temperature Readings: IoT Devices (Kaggle, autor atulanandjha), arquivo `IOT-temp.csv`, com 6,9 MB, **97.606 linhas e 5 colunas**. As leituras vão de 28/07/2018 a 08/12/2018 e vêm de uma única sala (*Room Admin*), com sensores interno e externo.

Coluna	Tipo	Conteúdo	Observações
id	texto	identificador da leitura	97.605 valores únicos (1 repetido)
room_id/id	texto	sala	um único valor: Room Admin
noted_date	texto	data e hora	formato DD-MM-YYYY HH:MM
temp	inteiro	temperatura (°C)	mínimo 21, máximo 51
out/in	texto	ambiente	Out: 77.261 · In: 20.345

O dataset não contém valores nulos. O exemplo do enunciado usa `device_id`, mas essa coluna não existe; por isso as análises comparam os ambientes interno e externo.

7. Processamento

O script `load_data.py` aplica as etapas abaixo e informa cada contagem no terminal:

- renomeia as colunas para nomes válidos em SQL (`out/in` → `location`, por exemplo);
- converte `noted_date` com formato explícito `%d-%m-%Y %H:%M`, evitando trocar dia por mês;
- converte a temperatura para número e remove linhas nulas ou não conversíveis (0 encontradas);
- valida o domínio: ambiente apenas In/Out e temperatura entre −50 e 100 °C (0 inválidas);
- remove duplicidades por linha idêntica e por id (1 encontrada).

Resultado: **97.606 linhas lidas → 97.605 registros válidos inseridos**. A carga roda numa única transação (`TRUNCATE` seguido de inserção), então executar o script de novo não duplica dados; isso foi testado rodando a carga duas vezes.

8. PostgreSQL

O banco roda na imagem oficial `postgres:16`, com volume persistente e healthcheck definidos no `docker-compose.yml`. A tabela criada é:

```
CREATE TABLE temperature_readings (
    reading_id  TEXT PRIMARY KEY,
    room_id     TEXT      NOT NULL,
    noted_at    TIMESTAMP NOT NULL,   -- indexada
    temperature SMALLINT  NOT NULL,
    location    TEXT      NOT NULL CHECK (location IN ('In','Out'))
);
```

A validação no banco confirmou 97.605 registros, nenhum valor nulo, temperaturas de 21 a 51 °C (média de 35,05 °C) e o período de 28/07/2018 07:06 a 08/12/2018 09:30.

9. Views SQL

9.1 temperatura_media_ambiente

Agrupa por ambiente e calcula quantidade, média, mínima, máxima e desvio padrão. Serve para comparar o comportamento térmico dentro e fora da sala.

```
SELECT location AS ambiente, COUNT(*) AS quantidade_leituras,
       ROUND(AVG(temperature)::numeric,2) AS temperatura_media,
       MIN(temperature), MAX(temperature),
       ROUND(STDDEV(temperature)::numeric,2) AS desvio_padrao
FROM temperature_readings GROUP BY location;
```

Ambiente	Leituras	Média	Mín.	Máx.	Desvio
Out	77.260	36,27 °C	24	51	5,72
In	20.345	30,45 °C	21	41	2,24

9.2 leituras_por_hora

Para cada hora do dia (0–23) e ambiente, conta as leituras e calcula a temperatura média. Mostra quando os sensores mais transmitem e como a temperatura varia ao longo do dia.

```
SELECT EXTRACT(HOUR FROM noted_at)::int AS hora, location AS ambiente,
       COUNT(*) AS quantidade_leituras, ROUND(AVG(temperature)::numeric,2) AS
temperatura_media
FROM temperature_readings GROUP BY 1, 2 ORDER BY hora, ambiente;
```

9.3 temperatura_por_dia

Para cada dia e ambiente, traz mínima, máxima, média e quantidade de leituras. Permite acompanhar a evolução no tempo e identificar dias atípicos ou sem coleta.

```
SELECT DATE(noted_at) AS data, location AS ambiente,
       MIN(temperature), MAX(temperature), ROUND(AVG(temperature)::numeric,2),
COUNT(*)
FROM temperature_readings GROUP BY 1, 2 ORDER BY data, ambiente;
```

10. Dashboard

O dashboard tem filtros de ambiente, período e faixa de temperatura, quatro KPIs no topo e quatro gráficos interativos.

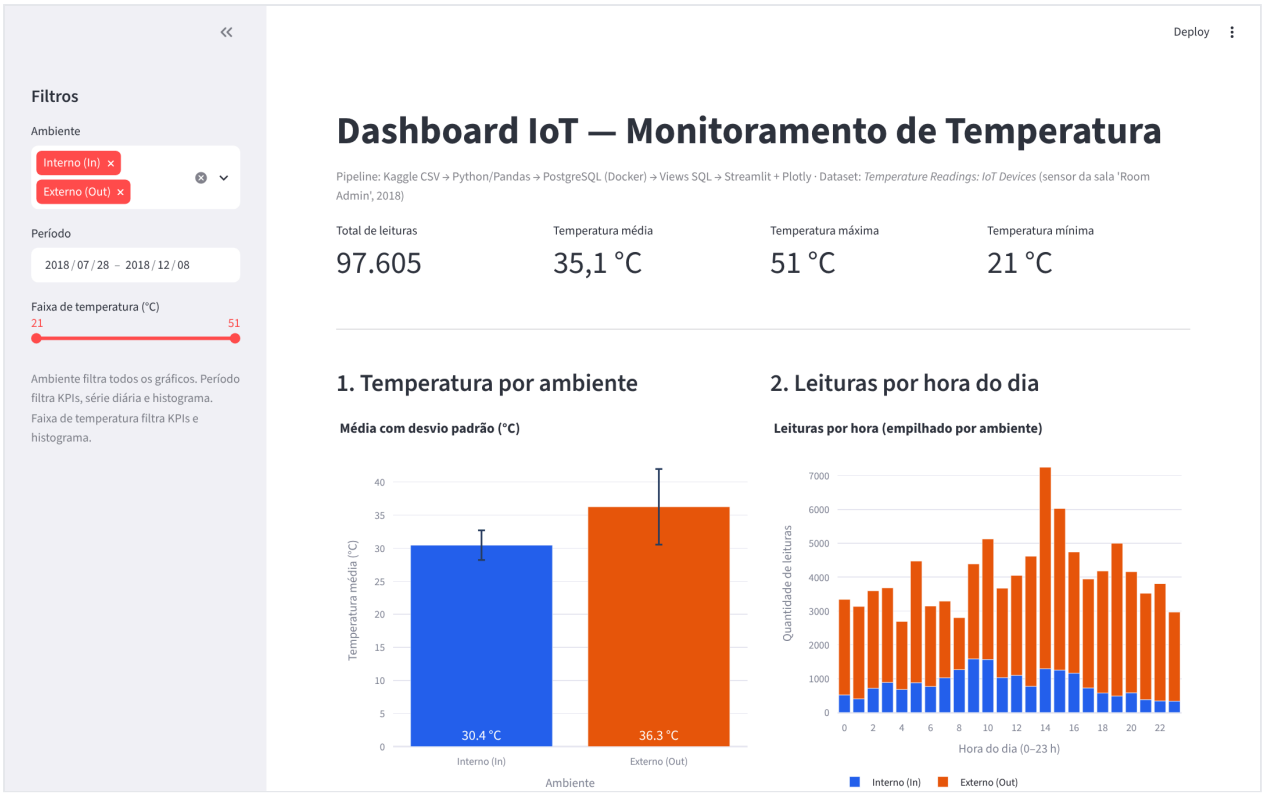


Figura 1 – Visão geral: filtros, KPIs e os dois primeiros gráficos.

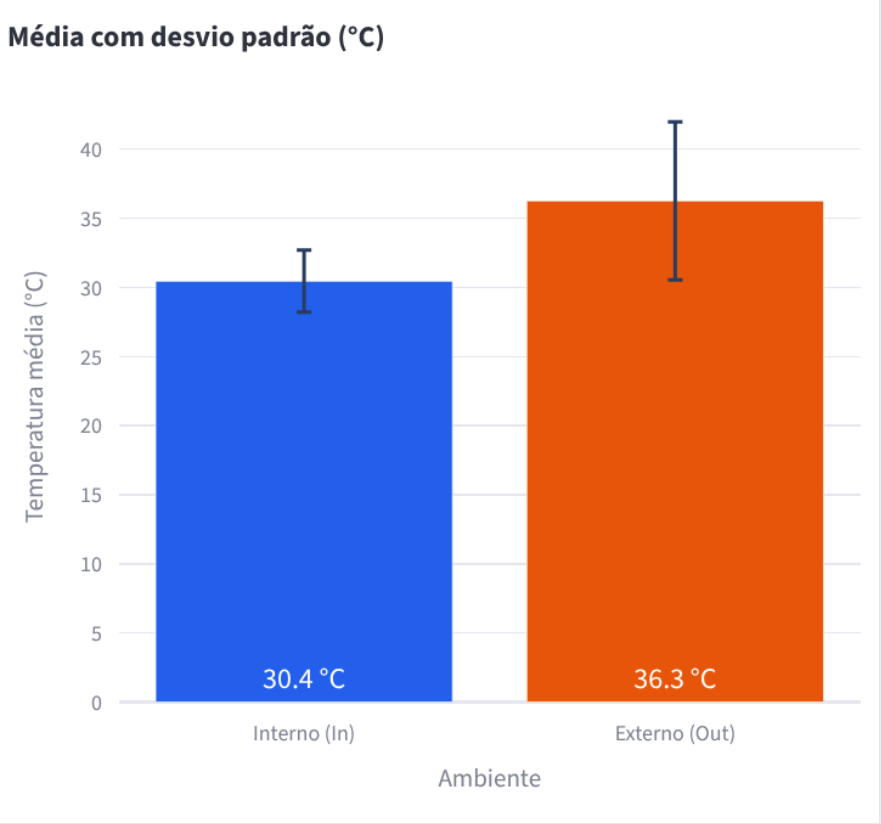


Figura 2 – Temperatura média por ambiente, com desvio padrão (view 1).

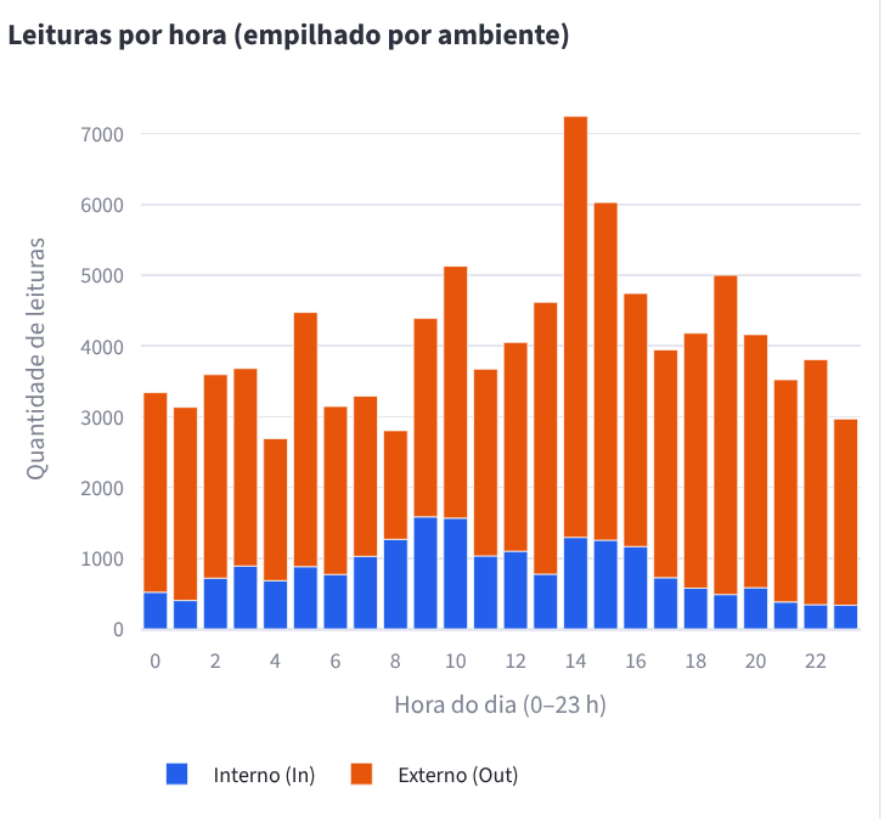


Figura 3 – Leituras por hora do dia, por ambiente (view 2).

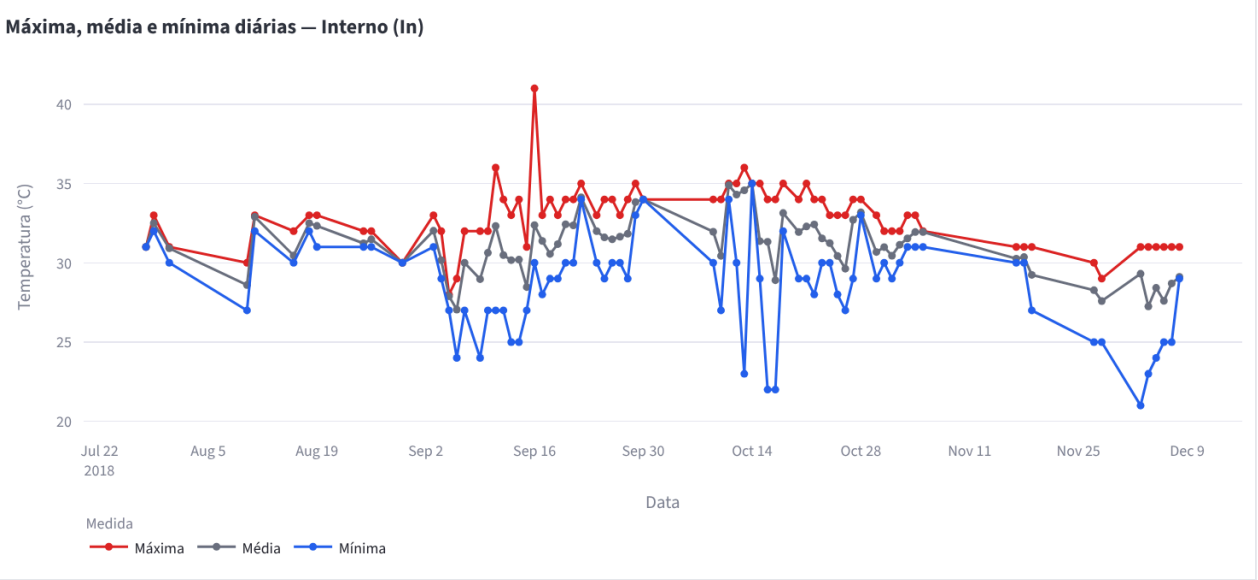


Figura 4 – Máxima, média e mínima diárias (view 3).

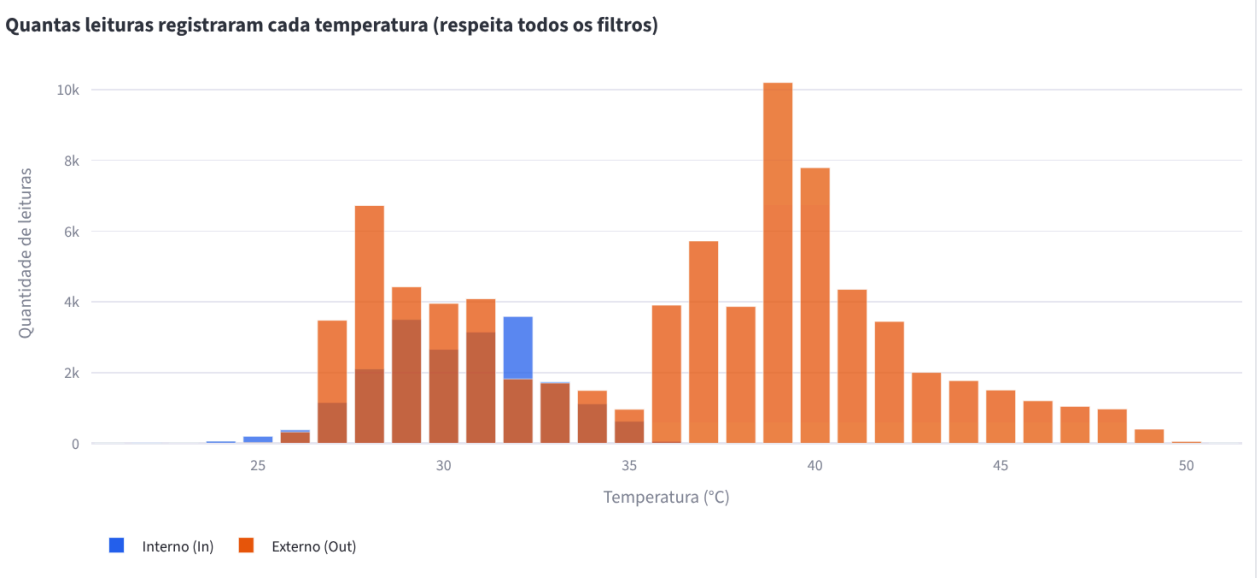


Figura 5 – Distribuição das temperaturas por ambiente.

11. Insights

1. **O ambiente externo é mais quente e mais instável:** média de 36,3 °C contra 30,4 °C no interno, com desvio padrão de 5,7 °C contra 2,2 °C. A sala amortece as variações.
2. **Há uma mudança de patamar a partir de outubro:** a média externa subiu de 31,2 °C em setembro para 39,9 °C em outubro e 40,2 °C em novembro. Os dias externos mais quentes foram 28 a 30/09/2018, com média de até 48,4 °C e máxima de 51 °C.
3. **79,2% das leituras são externas** (77.260 de 97.605), então médias gerais sem separar ambiente ficam enviesadas para o lado externo.
4. **A coleta é irregular:** houve leituras em apenas 86 dos 134 dias, e o dia 17/10/2018 sozinho concentra 20.564 leituras (21% do total), o que sugere falhas de conexão ou envio em lote.
5. **O pico de transmissão é às 14h** (7.248 leituras), e o menor volume é às 4h (2.690).
6. **Possível fuso horário diferente:** a média externa por hora é maior às 5h–6h (cerca de 39,5 °C) e menor às 19h (33,6 °C), o oposto do ciclo solar esperado. Isso indica que os horários provavelmente estão em outro fuso (por exemplo, UTC) e precisa ser confirmado antes de conclusões por hora.

12. Aplicação prática

Em um ambiente real, a mesma arquitetura pode monitorar data centers, câmaras frias, estufas ou salas de servidores. As mudanças principais seriam:

- trocar o CSV por ingestão contínua (MQTT ou API) gravando no PostgreSQL ou num banco de séries temporais (TimescaleDB);
- criar alertas quando a temperatura sair de uma faixa definida ou quando um sensor parar de enviar dados, problema que este dataset mostrou ser comum;
- padronizar o fuso horário na origem (UTC no banco, horário local só na visualização);
- usar o histórico para modelos de previsão e detecção de anomalias, por exemplo para antecipar picos de calor e acionar a climatização.

13. Conclusão

O projeto mostrou na prática as etapas de um pipeline de dados IoT: ingestão de uma fonte externa, tratamento e validação, armazenamento num banco em contêiner, modelagem de consultas com views e visualização interativa. Analisar o schema real antes de codificar foi decisivo, porque o dataset não tinha a coluna `device_id` do exemplo e o formato de data exigia conversão explícita. Os dados também revelaram problemas típicos de IoT, como coleta irregular e possível inconsistência de fuso horário, que reforçam a importância da qualidade de dados e do monitoramento dos próprios

sensores. O uso de Docker e de um README detalhado torna o trabalho reproduzível por qualquer pessoa.

Repositório: [LINK DO GITHUB] · **Vídeo:** [LINK DO VÍDEO]